Architectural Justification

Lab 3 - Component Modelling & Architectural Pattern Selection

Name:	Akshat Sharma	**SRN:**	PES1UG24CS042
Section:	5A	**Problem Statement:**	#42 - Internal Microservice Catalog & Health Portal

Architecture Selection: We chose **Microservices Architecture** for the Internal Microservice Catalog & Health Portal system.

Reason 1

The health-check requirement (FR-001) pings every registered microservice every 30 seconds and keeps recalculating a rolling 24-hour uptime number. As more services get added to the catalog this job keeps growing, but it has nothing to do with how often someone registers a new service (FR-002) or searches the docs (FR-005). If health monitoring is its own service, it can be scaled up on its own and restarted or redeployed without breaking the rest of the portal.

Reason 2

The five features are pretty different from each other technically - the dependency graph (FR-003) needs graph logic, the docs search (FR-005) needs indexing/searching over OpenAPI files, and the notification service (FR-004) is basically just a small service that fires off Slack/email alerts. Keeping them as separate microservices means each one can be updated or redeployed on its own schedule instead of having to release the whole system together every time one part changes.

Security Advantage

Since every request goes through one API Gateway, access rules can be set per-service instead of for the whole app. Only DevOps Engineers can call the Catalog Service's register/update endpoints, while something like the dependency graph or docs search can be left more open for read access. This also means if one service gets compromised, it doesn't automatically expose the others - e.g. a bug in the Docs Aggregator can't be used to get into the Health Monitor's DB credentials, since they run as separate services.

Performance Benefit

Health Monitor's 30-second polling loop is the heaviest and most time-sensitive part of the whole system, especially as more microservices get registered. Because it runs as its own service, it can be scaled independently instead of competing for CPU/memory with lighter, less frequent operations like docs search or registering a new service - so probe latency stays low and availability numbers stay accurate even as the catalog grows.